C++20 Standard Library Additions - New Tools for Modern Development

Theory, Edge Cases & Code Examples

- ✓ Non-Owning View of Contiguous Memory
- ✓ Type-Safe String Formatting
- ✓ Self-Joining Thread with Cancellation
- ✓ Better Logging and Debugging
- ✓ Bit Manipulation Utilities (``)

→ Swipe to learn →

C++20 Standard Library Additions - New Tools for Modern Development

Theory

📌 1. `std::span` - Non-Owning View of Contiguous Memory (1/2)

The Problem:

```
// ❌ C++17: Many ways to pass arrays, inconsistent
void process_ints(int* data, size_t size);        // C-style
void process_vector(std::vector<int>& vec);        // Vector-specific
void process_array(std::array<int, 10>& arr);     // Array-specific
template<size_t N>
void process_array_template(std::array<int, N>& arr);  // Template complexity
```

The C++20 Solution:

```
#include <span>

// ✅ C++20: Universal interface for contiguous sequences
void process(std::span<int> data) {
    for (int& val : data) {
        val *= 2;
    }
}

int main() {
    int c_array[] = {1, 2, 3};
    std::vector<int> vec = {4, 5, 6};
    std::array<int, 3> arr = {7, 8, 9};

    process(c_array);  // ✅ Works
    process(vec);      // ✅ Works
    process(arr);      // ✅ Works
}
```

Key Features:

- **Non-owning**: Like a reference, doesn't manage memory
- **Lightweight**: Just pointer + size (16 bytes on 64-bit)
- **Safe**: Bounds-checked access with `.at()` and `[]` in debug
- **Works with any contiguous container**: Arrays, vectors, std::array

C++20 Standard Library Additions - New Tools for Modern Development

Theory

📌 1. `std::span` - Non-Owning View of Contiguous Memory (2/2)

Dynamic vs Fixed Extent:

```
std::span<int> dynamic_span(data);        // Size known at runtime
std::span<int, 10> fixed_span(data, 10);  // Size known at compile-time

static_assert(sizeof(dynamic_span) == 16);  // pointer + size
static_assert(sizeof(fixed_span) == 8);     // just pointer (size is template arg)
```

Subspans:

```
std::vector<int> vec = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
std::span<int> full(vec);

auto first_5 = full.first(5);      // {1, 2, 3, 4, 5}
auto last_3 = full.last(3);        // {8, 9, 10}
auto middle = full.subspan(3, 4);  // {4, 5, 6, 7}
```

C++20 Standard Library Additions - New Tools for Modern Development

Theory

📌 2. `std::format` - Type-Safe String Formatting (1/2)

The Problem:

```
// ❌ C++17: printf is not type-safe, iostreams are verbose
int age = 30;
std::string name = "Alice";

// printf: Not type-safe, wrong format specifier = UB
printf("Hello, %s! You are %d years old.\n", name.c_str(), age);

// iostream: Verbose, hard to read
std::cout << "Hello, " << name << "! You are " << age << " years old.\n";
```

The C++20 Solution:

```
#include <format>

// ✅ C++20: Type-safe, Python-like formatting
std::string msg = std::format("Hello, {}! You are {} years old.", name, age);
// Output: "Hello, Alice! You are 30 years old."
```

C++20 Standard Library Additions - New Tools for Modern Development

Theory

📌 2. `std::format` - Type-Safe String Formatting (2/2)

Format Specifiers:

```
int num = 42;
double pi = 3.14159;

// Positional arguments
std::format("{0} + {1} = {2}", 10, 20, 30);  // "10 + 20 = 30"

// Width and alignment
std::format("{:10}", num);        // "        42" (right-aligned, width 10)
std::format("{:<10}", num);       // "42        " (left-aligned)
std::format("{:^10}", num);       // "    42    " (center-aligned)

// Number formatting
std::format("{:d}", num);         // "42" (decimal)
std::format("{:x}", num);         // "2a" (hexadecimal)
std::format("{:b}", num);         // "101010" (binary)
std::format("{:o}", num);         // "52" (octal)

// Floating-point
std::format("{:.2f}", pi);        // "3.14" (2 decimal places)
std::format("{:.5f}", pi);        // "3.14159"
std::format("{:e}", pi);          // "3.141590e+00" (scientific)

// Fill and align
std::format("{:*>10}", num);      // "********42" (fill with *, right-align)
std::format("{:0>5}", num);       // "00042" (zero-padding)
```

Performance:

- Faster than iostreams (no virtual calls)
- Comparable to snprintf, but type-safe
- Compile-time format string checking (with `std::format_string`)

C++20 Standard Library Additions - New Tools for Modern Development

Theory

📌 3. `std::jthread` - Self-Joining Thread with Cancellation (1/2)

The Problem:

```
// ❌ C++17: std::thread requires manual joining
std::thread t([]() {
    // Long-running task
});

// If you forget t.join() or t.detach(), program terminates!
// t destructor calls std::terminate() if thread is joinable
```

The C++20 Solution:

```
#include <thread>
#include <iostream>

// ✅ C++20: Automatic joining
{
    std::jthread t([]() {
        std::cout << "Working...\n";
    });
    // Automatically joins when t goes out of scope
}
```

📌 3. `std::jthread` - Self-Joining Thread with Cancellation (2/2)

Cooperative Cancellation with `std::stop_token`:

```
#include <thread>
#include <stop_token>
#include <chrono>

void worker(std::stop_token stoken) {
    while (!stoken.stop_requested()) {
        // Do work
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }
    std::cout << "Stopping gracefully\n";
}

int main() {
    std::jthread t(worker);

    std::this_thread::sleep_for(std::chrono::seconds(2));

    t.request_stop();  // Signal thread to stop
    // t joins automatically when destroyed
}
```

`std::stop_source` and `std::stop_token`:

```
std::stop_source source;
std::stop_token token = source.get_token();

// Thread 1: Work until stopped
std::jthread t([token]() {
    while (!token.stop_requested()) {
        // Work...
    }
});

// Thread 2: Request stop
source.request_stop();
```

C++20 Standard Library Additions - New Tools for Modern Development

Theory

📌 4. `std::source_location` - Better Logging and Debugging (1/2)

The Problem:

```
// ❌ C++17: Manual __FILE__ and __LINE__ macros
void log_error(const char* msg, const char* file, int line) {
    std::cout << file << ":" << line << ": " << msg << '\n';
}

#define LOG_ERROR(msg) log_error(msg, __FILE__, __LINE__)

LOG_ERROR("Something went wrong");  // Macro needed
```

The C++20 Solution:

```
#include <source_location>
#include <iostream>

// ✅ C++20: Automatic source location
void log_error(const char* msg,
               std::source_location loc = std::source_location::current()) {
    std::cout << loc.file_name() << ":"
              << loc.line() << ":"
              << loc.column() << " ["
              << loc.function_name() << "]: "
              << msg << '\n';
}

void my_function() {
    log_error("Error occurred");  // No macro needed!
}

/* Output:
example.cpp:15:5 [void my_function()]: Error occurred
*/
```

📌 4. `std::source_location` - Better Logging and Debugging (2/2)

Available Information:

```
auto loc = std::source_location::current();

loc.file_name();      // "example.cpp"
loc.function_name();  // "void my_function()"
loc.line();           // 42
loc.column();         // 10
```

C++20 Standard Library Additions - New Tools for Modern Development

Theory

📌 5. Bit Manipulation Utilities (``) (1/2)

std::bit_cast - Type-Punning:

```
#include <bit>
#include <cstdint>

// Safe type reinterpretation (replaces unsafe unions/memcpy)
float f = 3.14f;
uint32_t bits = std::bit_cast<uint32_t>(f);  // View float as uint32_t

// Convert back
float f2 = std::bit_cast<float>(bits);
```

Bit Counting:

```
#include <bit>

uint32_t value = 0b10110100;

std::popcount(value);         // 4 (number of 1 bits)
std::countl_zero(value);      // 0 (leading zeros)
std::countr_zero(value);      // 2 (trailing zeros)
std::countl_one(value);       // 0 (leading ones)
std::countr_one(value);       // 0 (trailing ones)
```

Bit Rotations:

```
std::rotl(value, 2);  // Rotate left by 2 bits
std::rotr(value, 2);  // Rotate right by 2 bits
```

Power of 2 Checks:

```
std::has_single_bit(8);   // true (2^3)
std::has_single_bit(7);   // false
std::bit_ceil(7);         // 8 (next power of 2)
std::bit_floor(7);        // 4 (previous power of 2)
std::bit_width(7);        // 3 (bits needed to represent)
```

📌 5. Bit Manipulation Utilities (``) (2/2)

Endianness:

```
#include <bit>

if constexpr (std::endian::native == std::endian::little) {
    // Little endian system (x86, ARM)
} else if constexpr (std::endian::native == std::endian::big) {
    // Big endian system (some mainframes)
}
```

C++20 Standard Library Additions - New Tools for Modern Development

Theory

📌 6. Calendar and Timezone (`` Improvements)

Date Handling:

```
#include <chrono>
#include <iostream>

using namespace std::chrono;

// Create dates
auto today = year_month_day{2024y / March / 24};
auto tomorrow = today + days{1};

// Date arithmetic
auto next_week = today + weeks{1};
auto last_month = today - months{1};

// Weekday queries
auto wd = weekday{today};
std::cout << (wd == Monday ? "Monday" : "Not Monday") << '\n';
```

Time Zones:

```
#include <chrono>

using namespace std::chrono;

// Get current time in different zones
auto utc_time = system_clock::now();
auto ny_tz = locate_zone("America/New_York");
auto tokyo_tz = locate_zone("Asia/Tokyo");

auto ny_time = zoned_time{ny_tz, utc_time};
auto tokyo_time = zoned_time{tokyo_tz, utc_time};
```

Duration Formatting:

```
using namespace std::chrono_literals;

auto duration = 1h + 30min + 45s;
std::cout << std::format("{:%H:%M:%S}", duration);  // "01:30:45"
```

C++20 Standard Library Additions - New Tools for Modern Development

Theory

📌 7. Mathematical Constants (``)

```
#include <numbers>
#include <iostream>

using namespace std::numbers;

std::cout << pi << '\n';           // 3.14159...
std::cout << e << '\n';            // 2.71828...
std::cout << sqrt2 << '\n';        // 1.41421...
std::cout << ln2 << '\n';          // 0.693147...
std::cout << log2e << '\n';        // 1.4427...
std::cout << phi << '\n';          // 1.61803... (golden ratio)

// Type-specific versions
std::numbers::pi_v<float>;         // float precision
std::numbers::pi_v<double>;        // double precision
std::numbers::pi_v<long double>;   // long double precision
```

C++20 Standard Library Additions - New Tools for Modern Development

Theory

📌 8. String Enhancements

`starts_with`, `ends_with`, `contains`:

```
#include <string>
#include <string_view>

std::string str = "Hello, World!";

str.starts_with("Hello");    // true
str.starts_with('H');         // true
str.ends_with("World!");      // true
str.ends_with('!');           // true
str.contains("lo, W");        // true (C++23, but mentioned for completeness)
```

std::string_view Improvements:

```
std::string_view sv = "example";
sv.starts_with("ex");  // true
sv.ends_with("ple");   // true
```

C++20 Standard Library Additions - New Tools for Modern Development

Theory

📌 9. Synchronization Primitives (1/2)

`std::latch` **- Single-Use Countdown:**

```
#include <latch>
#include <thread>
#include <vector>

std::latch work_done(3);  // Wait for 3 threads

void worker(int id) {
    // Do work
    std::cout << "Worker " << id << " done\n";
    work_done.count_down();  // Signal completion
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 3; ++i) {
        threads.emplace_back(worker, i);
    }

    work_done.wait();  // Wait for all workers
    std::cout << "All workers completed\n";

    for (auto& t : threads) t.join();
}
```

C++20 Standard Library Additions - New Tools for Modern Development

Theory

📌 9. Synchronization Primitives (2/2)

`std::barrier` **- Reusable Synchronization Point:**

```
#include <barrier>
#include <thread>

std::barrier sync_point(3, []() noexcept {
    std::cout << "All threads reached barrier\n";
});

void worker() {
    for (int i = 0; i < 5; ++i) {
        // Phase 1 work
        sync_point.arrive_and_wait();  // Wait for all threads

        // Phase 2 work
        sync_point.arrive_and_wait();  // Reusable!
    }
}
```

`std::counting_semaphore` **and** `std::binary_semaphore`**:**

```
#include <semaphore>
#include <thread>

std::counting_semaphore<3> resource(3);  // Max 3 concurrent accesses

void use_resource() {
    resource.acquire();  // Decrement (block if 0)

    // Use resource
    std::this_thread::sleep_for(std::chrono::seconds(1));

    resource.release();  // Increment
}

std::binary_semaphore signal(0);  // Like std::mutex, but lighter
```

C++20 Standard Library Additions - New Tools for Modern Development

Theory

📌 10. Other Notable Additions

`std::to_array` **- Create std::array from C array:**

```
auto arr = std::to_array({1, 2, 3, 4, 5});
// Type: std::array<int, 5>
```

`std::erase` / `std::erase_if` **- Uniform Erase:**

```
std::vector<int> vec = {1, 2, 3, 4, 5};
std::erase(vec, 3);              // Remove all 3s
std::erase_if(vec, [](int n) { return n % 2 == 0; });  // Remove evens
```

`std::ssize` **- Signed Size:**

```
std::vector<int> vec = {1, 2, 3};
auto size = std::ssize(vec);  // Returns signed (ptrdiff_t), not size_t
```

`std::midpoint` **- Overflow-Safe Midpoint:**

```
int a = INT_MAX, b = INT_MAX - 2;
auto mid = std::midpoint(a, b);  // Correctly computes midpoint without overflow
```

`std::lerp` **- Linear Interpolation:**

```
double start = 0.0, end = 10.0, t = 0.5;
double result = std::lerp(start, end, t);  // 5.0 (midpoint)
```

C++20 Standard Library Additions - New Tools for Modern Development

Edge Cases

📌 1. `std::span` Dangling Reference

```
std::span<int> create_span() {
    int arr[] = {1, 2, 3};
    return std::span(arr);  // ❌ Dangling! arr destroyed after return
}

// Solution: Return owning container or ensure lifetime
std::vector<int> create_vector() {
    return {1, 2, 3};  // ✅ OK
}
```

C++20 Standard Library Additions - New Tools for Modern Development

Edge Cases

📌 2. `std::format` Compile-Time Validation

```
int value = 42;

// ✅ OK: Format string checked at compile-time
std::format("{}", value);

// ❌ Compile error: Mismatched types
std::format("{:f}", value);  // {:f} expects floating-point, got int

// Runtime format string (less safe):
std::string fmt = "{}";
std::vformat(fmt, std::make_format_args(value));  // No compile-time check
```

C++20 Standard Library Additions - New Tools for Modern Development

Edge Cases

📌 3. `std::jthread` Callback Signature

```
// ❌ Wrong: No stop_token parameter
std::jthread t([]() {
    while (true) {  // Can't check for stop request
        // ...
    }
});

// ✅ Correct: Accept stop_token
std::jthread t([](std::stop_token stoken) {
    while (!stoken.stop_requested()) {
        // ...
    }
});
```

C++20 Standard Library Additions - New Tools for Modern Development

Edge Cases

📌 4. `std::bit_cast` Size Requirement

```
struct Small { char c; };  // 1 byte
struct Large { int i; };   // 4 bytes

// ❌ Compile error: Sizes don't match
auto x = std::bit_cast<Large>(Small{'a'});

// ✅ OK: Same size
uint32_t a = 42;
float f = std::bit_cast<float>(a);
static_assert(sizeof(uint32_t) == sizeof(float));
```

C++20 Standard Library Additions - New Tools for Modern Development

Code Examples

📌 1. Generic Array Processing with `std::span`

```
#include <span>
#include <vector>
#include <array>
#include <iostream>

// Universal function for any contiguous container
double average(std::span<const int> data) {
    if (data.empty()) return 0.0;

    int sum = 0;
    for (int val : data) {
        sum += val;
    }
    return static_cast<double>(sum) / data.size();
}

int main() {
    int c_array[] = {1, 2, 3, 4, 5};
    std::vector<int> vec = {10, 20, 30};
    std::array<int, 4> arr = {100, 200, 300, 400};

    std::cout << average(c_array) << '\n';  // 3
    std::cout << average(vec) << '\n';      // 20
    std::cout << average(arr) << '\n';      // 250
}
```

C++20 Standard Library Additions - New Tools for Modern Development

Code Examples

📌 2. Structured Logging with `std::source_location`

```
#include <source_location>
#include <iostream>
#include <string>
#include <format>

enum class LogLevel { INFO, WARNING, ERROR };

void log(LogLevel level, const std::string& msg,
         std::source_location loc = std::source_location::current()) {
    const char* level_str[] = {"INFO", "WARNING", "ERROR"};

    std::cout << std::format("[{}] {}:{} in {}: {}",
                             level_str[static_cast<int>(level)],
                             loc.file_name(),
                             loc.line(),
                             loc.function_name(),
                             msg) << '\n';
}

void process_data() {
    log(LogLevel::INFO, "Processing started");
    // ...
    log(LogLevel::ERROR, "Invalid data encountered");
}

int main() {
    process_data();
}

/* Output:
[INFO] example.cpp:18 in void process_data(): Processing started
[ERROR] example.cpp:20 in void process_data(): Invalid data encountered
*/
```

C++20 Standard Library Additions - New Tools for Modern Development

Code Examples

📌 3. Parallel Task Pool with `std::jthread` (1/2)

```
#include <thread>
#include <vector>
#include <queue>
#include <mutex>
#include <condition_variable>
#include <functional>

class ThreadPool {
    std::vector<std::jthread> workers_;
    std::queue<std::function<void()>> tasks_;
    std::mutex mutex_;
    std::condition_variable cv_;

public:
    ThreadPool(size_t num_threads) {
        for (size_t i = 0; i < num_threads; ++i) {
            workers_.emplace_back([this](std::stop_token stoken) {
                while (!stoken.stop_requested()) {
                    std::function<void()> task;
                    {
                        std::unique_lock lock(mutex_);
                        cv_.wait(lock, [this, &stoken] {
                            return !tasks_.empty() || stoken.stop_requested();
                        });

                        if (stoken.stop_requested()) break;

                        task = std::move(tasks_.front());
                        tasks_.pop();
                    }
                    task();
                }
            });
        }
    }

    void submit(std::function<void()> task) {
        {
            std::lock_guard lock(mutex_);
// ...continues on next slide
```

C++20 Standard Library Additions - New Tools for Modern Development

Code Examples

📌 3. Parallel Task Pool with `std::jthread` (2/2)

[Code continues across slides]

```
// ...continued from previous slide
            tasks_.push(std::move(task));
        }
        cv_.notify_one();
    }

    ~ThreadPool() {
        for (auto& worker : workers_) {
            worker.request_stop();
        }
        cv_.notify_all();
    }  // Workers automatically join
};

int main() {
    ThreadPool pool(4);

    for (int i = 0; i < 10; ++i) {
        pool.submit([i] {
            std::cout << "Task " << i << " executed\n";
        });
    }

    std::this_thread::sleep_for(std::chrono::seconds(2));
}  // Pool destruction automatically stops and joins all threads
```

C++20 Standard Library Additions - New Tools for Modern Development

Code Examples

📌 4. Safe Binary Serialization with `std::bit_cast`

```
#include <bit>
#include <cstdint>
#include <vector>

struct Header {
    uint32_t magic;
    uint32_t version;
    uint64_t timestamp;
};

std::vector<uint8_t> serialize(const Header& header) {
    static_assert(sizeof(Header) == 16);

    std::vector<uint8_t> bytes(sizeof(Header));
    std::memcpy(bytes.data(), &header, sizeof(Header));
    return bytes;
}

Header deserialize(const std::vector<uint8_t>& bytes) {
    Header header;
    std::memcpy(&header, bytes.data(), sizeof(Header));
    return header;
}

// Or using bit_cast (if types align):
std::array<uint8_t, 16> serialize_with_bitcast(const Header& header) {
    return std::bit_cast<std::array<uint8_t, 16>>(header);
}
```

C++20 Standard Library Additions - New Tools for Modern Development

Code Examples

📌 5. Pipeline Coordination with `std::barrier`

```
#include <barrier>
#include <thread>
#include <vector>
#include <iostream>

constexpr int NUM_THREADS = 4;
constexpr int NUM_STAGES = 3;

std::barrier stage_barrier(NUM_THREADS, []() noexcept {
    std::cout << "Stage completed by all threads\n";
});

void pipeline_worker(int id) {
    for (int stage = 0; stage < NUM_STAGES; ++stage) {
        std::cout << "Thread " << id << " working on stage " << stage << '\n';
        std::this_thread::sleep_for(std::chrono::milliseconds(100 * (id + 1)));

        stage_barrier.arrive_and_wait();  // Wait for all threads
    }
}

int main() {
    std::vector<std::jthread> threads;
    for (int i = 0; i < NUM_THREADS; ++i) {
        threads.emplace_back(pipeline_worker, i);
    }
}  // Automatic joining
```

C++20 Standard Library Additions - New Tools for Modern Development

Code Examples

📌 6. Date Arithmetic for Business Logic

```
#include <chrono>
#include <iostream>

using namespace std::chrono;

// Calculate next business day (skip weekends)
year_month_day next_business_day(year_month_day date) {
    date = sys_days{date} + days{1};  // Next day

    weekday wd{date};
    if (wd == Saturday) {
        date = sys_days{date} + days{2};  // Skip to Monday
    } else if (wd == Sunday) {
        date = sys_days{date} + days{1};  // Skip to Monday
    }

    return date;
}

int main() {
    // Friday
    auto friday = 2024y / March / 22;
    std::cout << "Friday: " << friday << '\n';

    // Next business day (should be Monday)
    auto monday = next_business_day(friday);
    std::cout << "Next business day: " << monday << '\n';  // 2024-03-25

    // Weekend days
    auto saturday = 2024y / March / 23;
    auto next_biz = next_business_day(saturday);
    std::cout << "After Saturday: " << next_biz << '\n';   // 2024-03-25 (Monday)
}
```

C++20 Standard Library Additions - New Tools for Modern Development

Code Examples

📌 7. Timezone-Aware Event Scheduling

```
#include <chrono>
#include <iostream>
#include <format>

using namespace std::chrono;

struct Meeting {
    std::string title;
    zoned_time<system_clock::duration> time;
};

void print_meeting(const Meeting& meeting, const std::string& user_timezone) {
    // Convert to user's timezone
    auto user_tz = locate_zone(user_timezone);
    auto user_time = zoned_time{user_tz, meeting.time.get_sys_time()};

    std::cout << std::format("{}: {} in your timezone\n",
                             meeting.title,
                             user_time);
}

int main() {
    // Schedule meeting at 10:00 AM New York time
    auto ny_tz = locate_zone("America/New_York");
    auto meeting_time = zoned_time{ny_tz, local_days{2024y/March/24} + 10h};

    Meeting meeting{"Team Standup", meeting_time};

    // Show for users in different timezones
    print_meeting(meeting, "America/Los_Angeles");  // 7:00 AM PST
    print_meeting(meeting, "Asia/Tokyo");           // 11:00 PM JST
    print_meeting(meeting, "Europe/London");        // 2:00 PM GMT
}
```

C++20 Standard Library Additions - New Tools for Modern Development

Code Examples

📌 8. Bit Manipulation for Feature Flags (1/2)

```
#include <bit>
#include <cstdint>
#include <iostream>

enum class Features : uint32_t {
    None         = 0,
    Logging      = 1 << 0,  // Bit 0
    Analytics    = 1 << 1,  // Bit 1
    DarkMode     = 1 << 2,  // Bit 2
    Notifications = 1 << 3,  // Bit 3
    All          = 0xFFFFFFFF
};

class FeatureFlags {
    uint32_t flags_ = 0;

public:
    void enable(Features feature) {
        flags_ |= static_cast<uint32_t>(feature);
    }

    void disable(Features feature) {
        flags_ &= ~static_cast<uint32_t>(feature);
    }

    bool is_enabled(Features feature) const {
        return (flags_ & static_cast<uint32_t>(feature)) != 0;
    }

    int enabled_count() const {
        return std::popcount(flags_);  // C++20: Count set bits
    }

    void print() const {
        std::cout << "Enabled features: " << enabled_count() << '\n';
        std::cout << "Binary: " << std::format("{:032b}", flags_) << '\n';
    }
};

// ...continues on next slide
```

📌 8. Bit Manipulation for Feature Flags (2/2)

[Code continues across slides]

```
// ...continued from previous slide
int main() {
    FeatureFlags config;

    config.enable(Features::Logging);
    config.enable(Features::DarkMode);
    config.enable(Features::Notifications);

    config.print();
    // Enabled features: 3
    // Binary: 00000000000000000000000000001101

    std::cout << "Dark mode: " << config.is_enabled(Features::DarkMode) << '\n';  // true
    std::cout << "Analytics: " << config.is_enabled(Features::Analytics) << '\n';  // false
}
```

C++20 Standard Library Additions - New Tools for Modern Development

Code Examples

📌 9. Format-Based Report Generation (1/2)

```
#include <format>
#include <vector>
#include <string>
#include <iostream>

struct SalesRecord {
    std::string product;
    int quantity;
    double price;
};

std::string generate_sales_report(const std::vector<SalesRecord>& records) {
    std::string report;

    // Header
    report += std::format("{:=^60}\n", " SALES REPORT ");
    report += std::format("{:<20} {:>10} {:>12} {:>12}\n",
                          "Product", "Quantity", "Price", "Total");
    report += std::format("{:-^60}\n", "");

    // Data rows
    double grand_total = 0.0;
    for (const auto& rec : records) {
        double total = rec.quantity * rec.price;
        grand_total += total;

        report += std::format("{:<20} {:>10} ${:>10.2f} ${:>10.2f}\n",
                              rec.product,
                              rec.quantity,
                              rec.price,
                              total);
    }

    // Footer
    report += std::format("{:-^60}\n", "");
    report += std::format("{:<43} ${:>10.2f}\n", "GRAND TOTAL", grand_total);
    report += std::format("{:=^60}\n", "");

    return report;
// ...continues on next slide
```

📌 9. Format-Based Report Generation (2/2)

[Code continues across slides]

```
// ...continued from previous slide
}

int main() {
    std::vector<SalesRecord> sales = {
        {"Widget A", 100, 19.99},
        {"Widget B", 50, 29.99},
        {"Widget C", 75, 15.50}
    };

    std::cout << generate_sales_report(sales);
}

/* Output:
==================== SALES REPORT =====================
Product                Quantity        Price        Total
------------------------------------------------------------
Widget A                    100     $    19.99  $  1999.00
Widget B                     50     $    29.99  $  1499.50
Widget C                     75     $    15.50  $  1162.50
------------------------------------------------------------
GRAND TOTAL                                     $  4661.00
============================================================
*/
```

C++20 Standard Library Additions - New Tools for Modern Development

Code Examples

📌 10. Semaphore-Based Connection Pool (1/2)

```
#include <semaphore>
#include <thread>
#include <vector>
#include <iostream>
#include <chrono>

class DatabaseConnection {
    int id_;
public:
    explicit DatabaseConnection(int id) : id_(id) {}
    void execute_query(const std::string& query) {
        std::cout << "Connection " << id_ << " executing: " << query << '\n';
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }
};

class ConnectionPool {
    std::vector<DatabaseConnection> connections_;
    std::counting_semaphore<10> available_;
    int next_connection_ = 0;
    std::mutex mutex_;

public:
    ConnectionPool(size_t size) : available_(size) {
        for (size_t i = 0; i < size; ++i) {
            connections_.emplace_back(i);
        }
    }

    class Connection {
        ConnectionPool* pool_;
        DatabaseConnection* conn_;

    public:
        Connection(ConnectionPool* pool, DatabaseConnection* conn)
            : pool_(pool), conn_(conn) {}

        ~Connection() {
            if (conn_) pool_->release();
// ...continues on next slide
```

📌 10. Semaphore-Based Connection Pool (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        }

        DatabaseConnection* operator->() { return conn_; }
    };

    Connection acquire() {
        available_.acquire();  // Wait if no connections available

        std::lock_guard lock(mutex_);
        DatabaseConnection* conn = &connections_[next_connection_];
        next_connection_ = (next_connection_ + 1) % connections_.size();

        return Connection{this, conn};
    }

private:
    void release() {
        available_.release();
    }
};

int main() {
    ConnectionPool pool(3);  // Max 3 concurrent connections

    std::vector<std::jthread> clients;
    for (int i = 0; i < 10; ++i) {
        clients.emplace_back([&pool, i] {
            auto conn = pool.acquire();  // Blocks if pool full
            conn->execute_query(std::format("SELECT * FROM table{}", i));
        });
    }

    // All threads join automatically
}
```

What You Learned

Key Takeaways

- ✓ Non-Owning View of Contiguous Memory
- ✓ Type-Safe String Formatting
- ✓ Self-Joining Thread with Cancellation
- ✓ Better Logging and Debugging

→ *Swipe to learn* →